

DeBERTa: Disentangled Attention.

BERT bolts a position into each token’s vector and hopes attention can untangle “what” from “where”. DeBERTa refuses to mix them: every token carries *two* vectors — a content vector and a relative-position vector — and the attention score splits into three readable terms. Every number here — three score components for a token pair, their sum, the $1/\sqrt{3d}$ scaling, and a softmax row — is computed in full, nothing summarised or deferred.

THE EQUATION

$$A_{ij} = \underbrace{H_i W_{q,c} (H_j W_{k,c})^\top}_{\text{content} \rightarrow \text{content}} + \underbrace{H_i W_{q,c} (P_{i|j} W_{k,r})^\top}_{\text{content} \rightarrow \text{position}} + \underbrace{P_{j|i} W_{q,r} (H_j W_{k,c})^\top}_{\text{position} \rightarrow \text{content}}$$

Worked example. Three tokens with content vectors in $\mathbb{R}^2$ and relative-position vectors keyed by $\delta = i - j$; all three score components for the pair $(i=3, j=1)$, the full 3×3 score matrix, $1/\sqrt{3d}$ scaling, and a softmax row.

Topic 1 of 2 in this module · companion to the course page · v1.0

Contents

1	Two vectors per token: content and position	2
2	Relative-distance bucketing	3
3	Worked example: three tokens in $\mathbb{R}^2$	3
4	Contrast with absolute positional embeddings	5
5	The enhanced mask decoder (briefly)	5
6	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 Two vectors per token: content and position

Standard BERT adds an *absolute* positional embedding to each token embedding *once*, at the input. From then on a single vector carries both meaning and place, and every attention head must disentangle them implicitly. DeBERTa (Decoding-enhanced BERT with disentangled attention) keeps them separate the whole way through: each token i owns a *content* vector H_i and, for any partner j , a *relative-position* vector $P_{i|j}$ that depends only on the signed distance $\delta = i - j$.

$$A_{ij} = H_i W_{q,c} (H_j W_{k,c})^\top + H_i W_{q,c} (P_{i|j} W_{k,r})^\top + P_{j|i} W_{q,r} (H_j W_{k,c})^\top \quad (1)$$

Four learned projections appear: content query/key $W_{q,c}, W_{k,c}$ and *relative* query/key $W_{q,r}, W_{k,r}$. The score is the sum of three cross-terms. There is a fourth conceivable term, position-to-position $P_{i|j} W_{q,r} (P_{j|i} W_{k,r})^\top$, but it carries no token content — it is the same for every pair at a given distance — so DeBERTa *drops it as redundant*.

Symbol	Meaning	Shape
H_i	content vector of token i	vector in $\mathbb{R}^d$
$P_{i j}$	relative-position vector, query side, $\delta = i - j$	vector in $\mathbb{R}^d$
$P_{j i}$	relative-position vector, key side, $\delta = j - i$	vector in $\mathbb{R}^d$
$W_{q,c}, W_{k,c}$	content query / key projections	$\mathbb{R}^{d \times d}$
$W_{q,r}, W_{k,r}$	relative-position query / key projections	$\mathbb{R}^{d \times d}$
$\delta = i - j$	relative distance (bucketed)	integer
A_{ij}	disentangled attention score	scalar

“The cat that I saw” — the word *saw* attends to *cat* for two reasons: *what cat* is (a noun, the subject) and *where* it sits (a few tokens back). BERT crams both cues into one vector. DeBERTa keeps two ledgers: a content ledger (“is this the right kind of word?”) and a position ledger (“is it the right distance away?”), then adds the bills. The cross-terms in Eq. 1 are exactly “my content vs. your position” and “my position vs. your content”.

2 Relative-distance bucketing

The position vectors are indexed by $\delta = i - j$, not by absolute index, so a phrase keeps the same positional signature no matter where in the sentence it appears. Distances are *bucketed*: for $|\delta| < k$ the bucket is δ itself; beyond a threshold k the buckets grow logarithmically and saturate, so a single embedding table of size $2k$ covers arbitrarily long sequences. Concretely, with clip distance k ,

$$\delta(i, j) = \text{clip}(i - j, -k, +k), \quad P_{i|j} = E_{\delta(i,j)+k}^r,$$

where $E^r \in \mathbb{R}^{2k \times d}$ is the relative-position embedding table. Near distances get their own exact bucket; far ones share. Below we use k large enough that all of $\delta \in \{-2, -1, 0, 1, 2\}$ are exact.

3 Worked example: three tokens in $\mathbb{R}^2$

Let $d = 2$. Content vectors (rows) and the relative-position table keyed by δ :

$$H = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \quad (\text{rows } H_1, H_2, H_3), \quad \begin{aligned} P_{\delta=-2} &= [-1, -1] \\ P_{\delta=-1} &= [0, -1] \\ P_{\delta=0} &= [1, 0] \\ P_{\delta=1} &= [0, 1] \\ P_{\delta=2} &= [1, 1] \end{aligned}$$

$$W_{q,c} = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \quad W_{k,c} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_{q,r} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad W_{k,r} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}.$$

3.1 The three components for the pair ($i=3, j=1$)

Here $H_i = H_3 = [1, 1]$, $H_j = H_1 = [1, 0]$, and $\delta = i - j = 2$, so $P_{i|j} = P_{\delta=2} = [1, 1]$ and (key side) $\delta = j - i = -2$, so $P_{j|i} = P_{\delta=-2} = [-1, -1]$. Project:

$$\begin{aligned} \text{content query } q_c &= H_i W_{q,c} = [1, 1] \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} = [1, 2], \\ \text{content key } k_c &= H_j W_{k,c} = [1, 0] \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} = [2, 0], \\ \text{rel. key } k_r &= P_{i|j} W_{k,r} = [1, 1] \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = [2, 1], \\ \text{rel. query } q_r &= P_{j|i} W_{q,r} = [-1, -1] \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = [-1, -2]. \end{aligned}$$

Now the three dot products of Eq. 1:

$$\begin{aligned} c \rightarrow c : q_c \cdot k_c &= [1, 2] \cdot [2, 0] = 1 \cdot 2 + 2 \cdot 0 = \mathbf{2}, \\ c \rightarrow p : q_c \cdot k_r &= [1, 2] \cdot [2, 1] = 1 \cdot 2 + 2 \cdot 1 = \mathbf{4}, \\ p \rightarrow c : q_r \cdot k_c &= [-1, -2] \cdot [2, 0] = -1 \cdot 2 + (-2) \cdot 0 = \mathbf{-2}. \end{aligned}$$

$$A_{31} = 2 + 4 + (-2) = \mathbf{4}.$$

The position cues dominate the content cue here: a same-content match would have scored only 2, but the favourable $\delta = 2$ alignment lifts it.

3.2 The full score matrix

Repeating the three-term computation for all nine (i, j) pairs gives the components and their sum:

$$\underbrace{\begin{bmatrix} 2 & 0 & 2 \\ 0 & 2 & 2 \\ 2 & 2 & 4 \end{bmatrix}}_{c \rightarrow c} + \underbrace{\begin{bmatrix} 1 & -1 & -2 \\ 2 & 0 & -2 \\ 4 & 3 & 1 \end{bmatrix}}_{c \rightarrow p} + \underbrace{\begin{bmatrix} 2 & 1 & 4 \\ 0 & 1 & 1 \\ -2 & -1 & 3 \end{bmatrix}}_{p \rightarrow c} = \underbrace{\begin{bmatrix} 5 & 0 & 4 \\ 2 & 3 & 1 \\ 4 & 4 & 8 \end{bmatrix}}_A.$$

3.3 Scale by $1/\sqrt{3d}$

Because the score is a sum of *three* dot-product terms (not one), its variance is roughly tripled, so DeBERTa scales by $1/\sqrt{3d}$ rather than $1/\sqrt{d}$. With $d = 2$, $\sqrt{3d} = \sqrt{6} \approx 2.4495$. Row $i = 3$ of A is $[4, 4, 8]$, so the scaled logits are

$$\frac{[4, 4, 8]}{\sqrt{6}} = [1.63299, 1.63299, 3.26599].$$

3.4 Softmax the query-3 row

Exponentiate and normalise:

$$e^z = [5.11917, 5.11917, 26.20595], \quad \sum e^z = 36.44429,$$

$$\text{softmax}(z) = \left[\underbrace{0.14047}_{j=1}, \underbrace{0.14047}_{j=2}, \underbrace{0.71907}_{j=3} \right].$$

	$j=1$	$j=2$	$j=3$
attn row $i=3$	0.14047	0.14047	0.71907

(Intensity $\approx$ probability $\times 80$, capped at **indigo!80**.) Token 3 attends mostly to itself: it has the strongest content match *and* the $\delta = 0$ self-position bucket.

Disentangled attention is a sum of three dot products — content $\times$ content, content $\times$ position, position $\times$ content — using *relative* distances $\delta = i - j$ and a $1/\sqrt{3d}$ scale. Keeping “what” and “where” in separate vectors lets one set of position embeddings serve every absolute location, and lets a head learn “attend two tokens back” independent

of the words involved.

4 Contrast with absolute positional embeddings

1. **Where position enters.** BERT adds an absolute embedding to the token embedding *once* at the input; DeBERTa injects relative position into *every* attention layer, via the content-to-position and position-to-content terms.
2. **Relative vs. absolute.** A relative scheme is translation-invariant: “two tokens to the left” means the same in position 5 or position 500, so a single bucketed table generalises to longer sequences than seen in training.
3. **Three terms, not one.** Dropping the position-to-position term (constant per distance, content-free) keeps the cost near a vanilla head while still modelling both cross-interactions; the $1/\sqrt{3d}$ scale corrects for the summed variance.

Do not reuse the $1/\sqrt{d}$ scale from ordinary attention. With three additive score terms the pre-softmax logits have $\sim 3\times$ the variance; dividing by $\sqrt{d}$ instead of $\sqrt{3d}$ inflates the logits, over-sharpens the softmax, and destabilises early training. Equally, do not feed *absolute* indices into P : the position vectors must be keyed by the relative distance $\delta = i - j$, and note the asymmetry — the query side uses $P_{i|j}$ ($\delta = i - j$) while the key side uses $P_{j|i}$ ($\delta = j - i$).

5 The enhanced mask decoder (briefly)

Relative positions alone cannot resolve every ambiguity. “a new [MASK] store” vs. “a [MASK] new store” have the same *relative* layout around the mask, yet *store* and *new* are different predictions. DeBERTa’s *enhanced mask decoder* (EMD) therefore feeds the *absolute* positions back in — but only at the final layer, just before the masked-token softmax — so the bulk of the network reasons relatively while the decoder still knows exactly where each masked slot sits. It is a small head-stage addition, not a change to Eq. 1.

6 Where this sits in the track

DeBERTa is the encoder-architecture refinement of the BERT/RoBERTa line (Modules 1.1–1.2): same MLM pretraining objective, a sharper attention mechanism. The disentangled scores feed the same softmax and value aggregation as Module 0.1’s scaled dot-product attention — only the score *construction* changes. Topic 2 turns to ELECTRA, which keeps a conventional encoder but replaces the *objective* with replaced-token detection for far greater sample efficiency.

A Reproducible (NumPy)

Inputs: H , the δ -keyed table P , and $W_{q,c}, W_{k,c}, W_{q,r}, W_{k,r}$ as above ($d = 2$, 1-indexed tokens in code use 0-indexing). The snippet regenerates the three component matrices, A , the $1/\sqrt{3d}$ -scaled row $i=3$, and its softmax.

```

import numpy as np
H = np.array([[1.,0.],[0.,1.],[1.,1.]])          # content rows H1,H2,H3
P = {-2: [-1.,-1.], -1: [0.,-1.], 0: [1.,0.], 1: [0.,1.], 2: [1.,1.]}
P = {k:np.array(v) for k,v in P.items()}
Wqc=np.array([[1.,0.],[0.,2.]]) Wkc=np.array([[2.,0.],[0.,1.]])
Wqr=np.array([[1.,1.],[0.,1.]]) Wkr=np.array([[1.,0.],[1.,1.]])
d=2; T=3
c2c=np.zeros((T,T)); c2p=np.zeros((T,T)); p2c=np.zeros((T,T))
for i in range(T):
    for j in range(T):
        qc=H[i]@Wqc; kc=H[j]@Wkc
        c2c[i,j]=qc@kc
        c2p[i,j]=qc@(P[i-j]@Wkr)          # delta = i-j (query side)
        p2c[i,j]=(P[j-i]@Wqr)@kc         # delta = j-i (key side)
A = c2c + c2p + p2c
z = A[2]/np.sqrt(3*d)                     # row i=3 (0-indexed 2)
soft = np.exp(z)/np.exp(z).sum()
print(A[2])      # [4. 4. 8.]
print(z)         # [1.63299 1.63299 3.26599]
print(soft)      # [0.14047 0.14047 0.71907]

```